Engineering Take-Home Task: Singapore Housing Indicator Workflow

Zhou Mo

4 October 2026

1. What was built

A command-line workflow that takes a report date and, in one command, downloads seven public macro indicators that lead Singapore home prices and rents, stores the raw downloads as checksummed snapshots, validates and normalises them, computes period changes in the correct unit, and writes a report in which every figure is traced to the published observations behind it.

```
python -m housing_agent run --date 2026-10-04
```

No forecasting model is trained. I treated the task as two questions: which few series are worth watching and why, and how to make the path from the publisher to the report reliable and reproducible. The repository contains the code, 22 tests that run without network, the raw snapshots from a live run on 4 October 2026, and the report that run produced.

2. Indicator selection

I chose indicators by asking what moves demand for housing, what moves the cost of financing it, and what moves supply, and then kept one or two series per channel. I preferred series that are official, published on a fixed schedule, available through a documented API, and long enough to compare with a year earlier. Sale and rental markets are read separately because some indicators affect them differently.

Indicator (publisher)	Observed	Change in	Sale	Rent	Reason for inclusion
3-month compounded SORA (MAS)	Monthly	bp	−	n/u	Reference rate for most floating-rate home loans; sets instalments and borrowing capacity under the TDSR
CPI, All Items (DOS)	Monthly	%	+	+	Nominal anchor for rents and building costs; the variable MAS policy responds to
Nominal GDP (DOS)	Quarterly	% y/y	+	+	Income growth; corporate demand for expatriate housing
Resident unemployment rate, SA (MOM)	Quarterly	pp	−	−	Income security of the households that buy
Total population (DOS)	Annual	%	+	+	Basic demand; variation is mostly non-residents, who rent
Private non-landed supply pipeline (URA)	Quarterly	%	−	−	Completions two to four years ahead
Private residential vacancy rate (derived, URA)	Quarterly	pp	−	−	Slack in existing stock; most direct rental signal

The sign is the expected pressure when the indicator rises; "n/u" means not used. Three judgements are worth stating:

- **SORA is not used for rents.** Cheaper mortgages pull tenants into buying, which lowers rental demand, but they also lower landlords' costs. The net sign is unclear, so I left it out rather than assert one.
- **GDP is read year-on-year only.** The series is not seasonally adjusted, so the quarter-on-quarter change mostly reflects the season. The report prints it with a marker and does not interpret it.
- **Vacancy rate is derived.** URA publishes vacant units and available units; the workflow divides them and shows the division in the report.

All seven come from the SingStat Table Builder API, which republishes agency series with uniform metadata (unit, frequency, last-updated date). That keeps one parser and one validation path, at the cost of a single point of failure, discussed in Section 6.

3. Workflow design

Stage	What happens	Output kept
Fetch	HTTP GET per indicator, three attempts with backoff for timeouts, 5xx and 429	Exact response bytes plus URL, retrieval time, SHA-256. Never overwritten
Parse and validate	Select the series by number and row text; check frequency and unit against the registry; parse period keys; record non-numeric values	One validated series, or a precise error

Stage	What happens	Output kept
Analyse	Choose the latest observation usable at the report date; compute changes; check staleness, gaps, plausible range, revisions	Values, changes, quality flags
Narrate	Rule-based directional reading per indicator and market; optional LLM paragraph	Commentary and its provenance
Report	Render Markdown from the results only (no clock, no network)	`report.md`, `observations.csv`, `manifest.json`

Registry. Everything known about a series (table, row, unit, frequency, expected publication lag, plausible range, sign, rationale) is in `config/indicators.yaml`. Adding an indicator is a configuration change.

Units. Each indicator declares how its change is expressed: interest rates in basis points, rates and shares in percentage points, indices and levels in per cent. This is tested directly.

Changes are looked up by period, not by position. The month-on-month change for August looks up July. If July is absent the change is reported as unavailable. Taking "the previous row" would silently compare August with June.

Traceability. For each figure the report prints the calculation with the published values and the snapshot file and checksum, for example: year-on-year −37.8 bp = (1.1863 − 1.5638) × 100, from 2026 Aug = 1.1863 and 2025 Aug = 1.5638.

Reproducibility. `manifest.json` records which snapshots produced a report. `--from-manifest` rebuilds the report from exactly those files, verified by checksum, without network. A test confirms the rebuilt report is byte-identical even after newer, revised data has been downloaded.

Where the LLM sits. No figure in the report comes from a model. The directional readings are computed by rule. With `--llm`, a model rewrites those readings into one paragraph per market. The text is accepted only if every number in it also appears in the computed facts; otherwise the rule-based paragraph is used and the rejection is recorded in the report. I chose this because the valuable property of the report is that its numbers can be trusted, and an unconstrained model weakens that. The submitted sample report was produced without an API key, so its commentary is rule-based.

4. Reliability

Situation	Behaviour
Download fails	Retried; then the newest stored snapshot is used and the report says so. Exit code 1
HTTP 200 with an error body or non-JSON	Rejected; falls back as above. The bad payload stays on disk as evidence
No data at all for an indicator	Shown as UNAVAILABLE; the rest of the report is still produced. Exit code 2
Row numbering shifts at the source	Series is matched on row text as well as number; a mismatch is an error that lists the rows returned
Unit or frequency changes at the source	Fatal for that indicator, since every computed change would otherwise be wrong without any visible sign
Missing or non-numeric period	The change that needs it is shown as n/a and logged
Stale data	Each indicator has a maximum expected age; older data is flagged STALE in the summary table
Revised history	Each run is compared with the previous snapshot; revised periods are listed
Report for a past date	Only periods ended by that date, less the typical publication lag, are used

A report is always written and the exit code distinguishes "current", "degraded" and "incomplete", so a scheduler can alert while a reader still sees what is missing.

5. Testing, a failure, and the fix

The 22 tests use payloads with the real API structure and a fake HTTP session. They cover period parsing, unit handling, as-of selection, each row of the table in Section 4, byte-identical reproduction, and the LLM number check.

Failure on the first live run. The tests passed, but the first run against the real API returned exit code 2 with SORA unavailable. The SORA row was selected by a keyword pattern. The live table contains both overnight SORA (series 21) and 3-month compounded SORA (series 23), so two rows matched. The workflow refused to guess and printed the rows it received:

```
UNAVAILABLE sora  downloaded payload rejected: 2 rows match ... Rows returned: ...
21='Singapore Overnight Rate Average';
23='Compounded Singapore Overnight Rate Average (SORA) - 3 Month'
```

Two things follow. First, the guard did its job: picking the first match would have produced a plausible number from the wrong series. Second, the cause was a test fixture that was simpler than reality, with only one SORA row. The fix was to select series 23 with an exact text match, because the compounded rate is the one home loans reference, and to add the

second row to the fixture so the ambiguity is now exercised by the tests. The second run returned all seven indicators with exit code 0. Both steps are in the commit history.

A second issue found during development. The API's `limit` parameter counts cells across all rows of a table, not periods per series. For a table with many rows this truncates each series to a few periods, which would leave too little history for a year-on-year change. The request therefore filters to the one series needed; the live run returned full history (for example 788 monthly CPI observations from 1961). If history is ever short, the change is reported as unavailable and a `short_history` warning is logged, which is also tested.

Result of the live run, 4 October 2026. Five of seven indicators read as upward pressure on sale prices (lower SORA, higher CPI, GDP and population, lower vacancy) and two as downward (higher resident unemployment, larger supply pipeline). For rents, four of six read as upward. These are directional readings, not a forecast.

6. Limitations and next steps

- **Single upstream API.** A change to the SingStat interface affects all indicators at once. Snapshots keep past reports reproducible but not current. Next step: a second route per series, such as data.gov.sg or the agency's own API.
- **Back-dated reports are approximate.** They use today's vintage with a publication-lag rule. True point-in-time data exists only for dates on which a snapshot was taken, so the workflow should run on a schedule to build that archive.
- **Signs are judgements.** They are not estimated, and lags differ by indicator. Next step: check each sign and lag against the URA price and rental indices.
- **Coverage.** No HDB supply series and no household income series are included.
- **The LLM check is numeric.** It stops invented figures but not a misleading sentence that uses correct ones.

7. Use of AI tools

I used Claude (Anthropic) as a coding assistant throughout, in a chat session with file access to the project folder.

- **Delegated to the AI:** proposing the indicator set and its rationale; writing the Python modules, the tests, the README and the first draft of this report; looking up SingStat table identifiers and checking the API response format; diagnosing the SORA failure from the output I sent it and making the fix; preparing the initial commit history.
- **Done by me:** setting the scope and constraints, running the tests and the live workflow on my own machine, reporting the results back, committing the fix and the sample data, and reviewing the indicator choices, signs and wording.
- **How outputs were checked:** all 22 tests pass on my machine; the live run was repeated after the fix and returned exit code 0; figures in the sample report were compared with the values shown on the SingStat website for the same series and period; the calculation column in the report was recomputed by hand for a sample of figures.

Commits that include AI-written code carry a `Co-Authored-By` line. I am responsible for the submitted work.